1. Python SQLite Game Shop Tutorial

Contents

1. Python SQLite Game Shop Tutorial 1
SQL Online Tutorial 1
SQLite with Python Tutorials 2
SQLite and Python 2
Self-Contained 3
Zero-Configuration 3
Transactional 3
Single-Database 3
CRUD (DY) 3
Tutorial 1: Create a Database with DB Browser for SQLite 4
Do: Install SQLite DB Browser for SQLite 4
Do: Create Database 4
Tutorial 2: Create a Database with Python 7
Explanation: How to Connect to an SQLite Database in Python 7
Do: sqlite_tutorial_1_create_database.py 7
Assignment Submission 9

Time required: 60 minutes

- Follow all directions carefully and accurately.
- Think of the directions as minimum requirements.

SQL Online Tutorial

- https://www.w3schools.com/sql/sql_intro.asp
- https://www.w3schools.com/sql/sql_syntax.asp
- https://www.w3schools.com/sql/sql_create_db.asp

SQLite with Python Tutorials

- SQLite Databases with Python - Full Course – FreeCodeCamp.org
- https://www.sqlitetutorial.net

SQLite and Python

SQLite is a self-contained, file-based SQL database. SQLite comes bundled with Python and can be used in any of your Python applications without having to install any additional software.

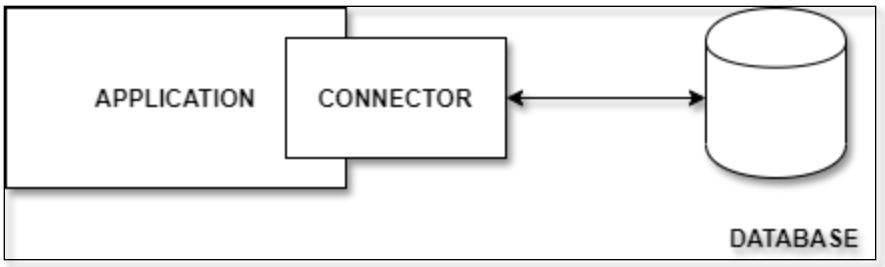
Databases offer numerous functionalities by which one can manage large amounts of information easily over the web and high-volume data input and output over a typical file such as a text file. SQL is a query language and is very popular in databases. Many websites use MySQL. SQLite is a "light" version that works over syntax very similar to SQL.

SQLite is a self-contained, high-reliability, embedded, full-featured, public-domain, SQL database engine. It is the most used database engine on the world wide web. Python has a library to access SQLite databases, called sqlite3, intended for working with SQLite.

SQLite has the following features.

1. Serverless
2. Self-Contained
3. Zero-Configuration
4. Transactional
5. Single-Database file

SQLite does not require a server to run. The SQLite database is joined with the application that accesses the database. SQLite database read and write directly from the database files stored on disk and applications interact with that SQLite database.



Self-Contained

SQLite does not need any external dependencies like an operating system or external library. This feature of SQLite helps especially in embedded devices like iPhones, Android phones, game consoles, handheld media players, etc.

Zero-Configuration

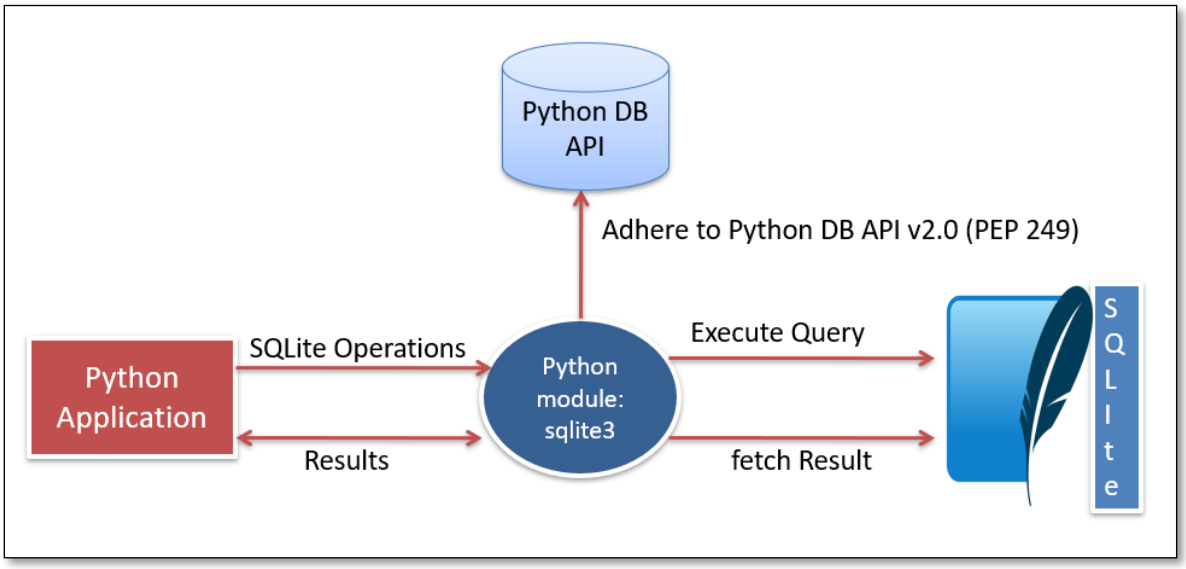
Zero-configuration means no setup or administration needed. Because of the serverless architecture, you don't need to "install" SQLite before using it. There is no server process that needs to be configured, started, and stopped.

Transactional

Transactional means they are atomic, consistent, isolated, and durable (ACID). All transactions in SQLite are fully ACID-compliant. In other words, all changes within a transaction take place completely or not at all even when an unexpected situation like application crash, power failure, or operating system crash occurs.

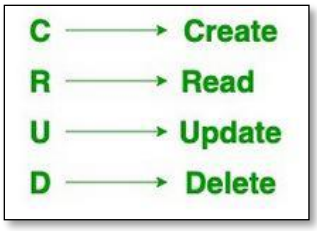
Single-Database

SQLite is a single database that means it allows a single database connection to access multiple database files simultaneously. These features bring many nice features like joining tables in different databases or copying data between databases in a single command. SQLite also uses dynamic types for tables. It means you can store any value in any column, regardless of the data type.



CRUD (DY)

A database provides four major operations, usually referred to as CRUD.



The corresponding SQL commands. Notice that SQL commands are typically in UPPER CASE. This is a convention; SQL does not require UPPER CASE. It makes the SQL commands easier to pick out of an SQL statement

CREATE: Create new tables and records

SELECT: Select all or selected fields and records

UPDATE: Modify existing tables and records

DELETE: Delete existing tables and records

Tutorial 1: Create a Database with DB Browser for SQLite

Do: Install SQLite DB Browser for SQLite

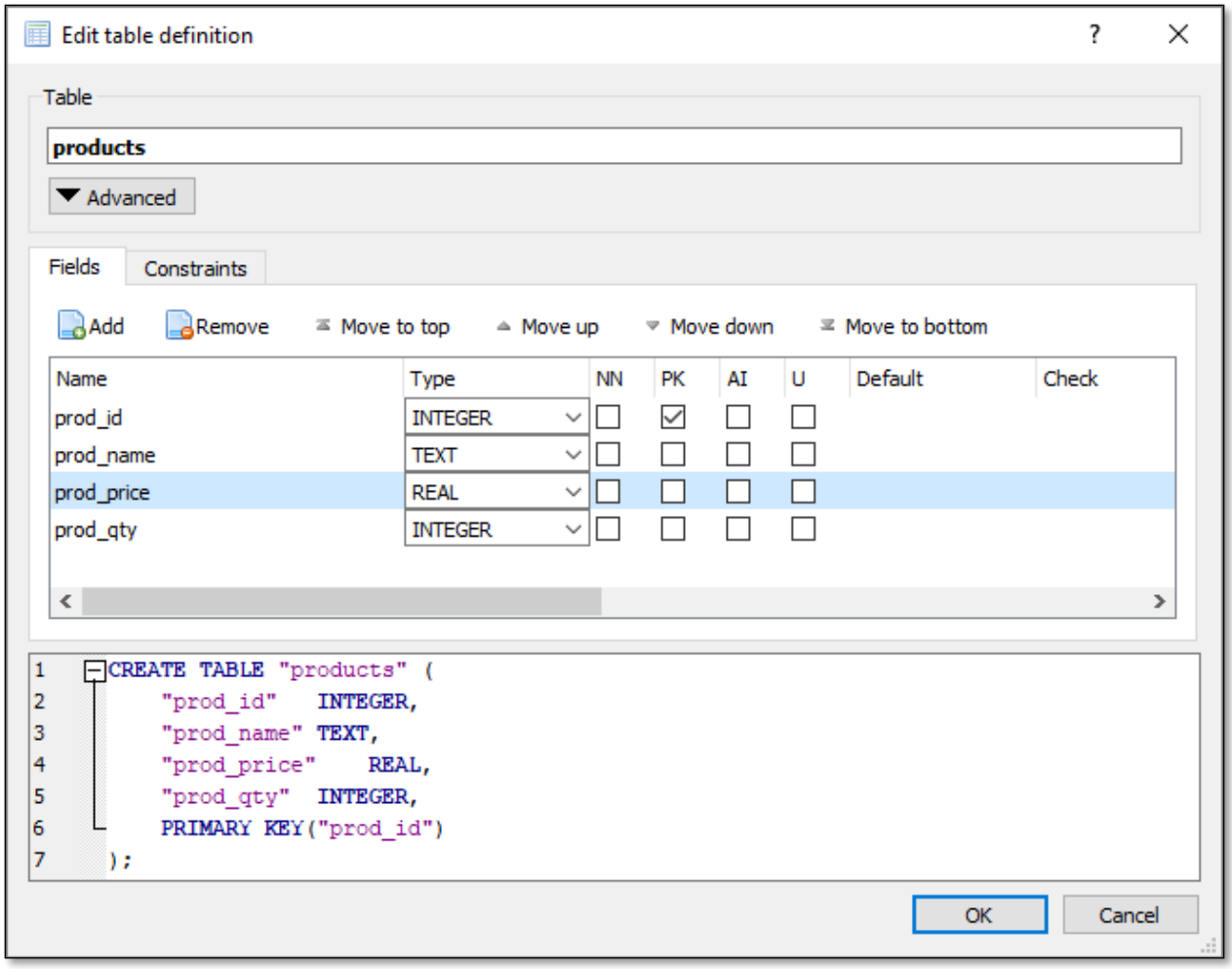
This is a handy tool to look at, troubleshoot, and manipulate your database. We will use it to learn the basics of SQL.

1. Go to https://sqlitebrowser.org
2. Go to the **Download** tab.
3. Download the **Windows PortableApp → DB Browser for SQLite - PortableApp**
4. Double Click the installation file. Click **Next**.
5. Click **Install**. Click **Finish**.
6. You will find a new folder: **SQLiteDatabaseBrowserPortable**
7. This folder can be moved anywhere, the program will work just fine.
8. In the folder you will find **SQLiteDatabaseBrowserPortable.exe**
9. Double Click the file. Click **OK** on the warning if there is one.

Do: Create Database

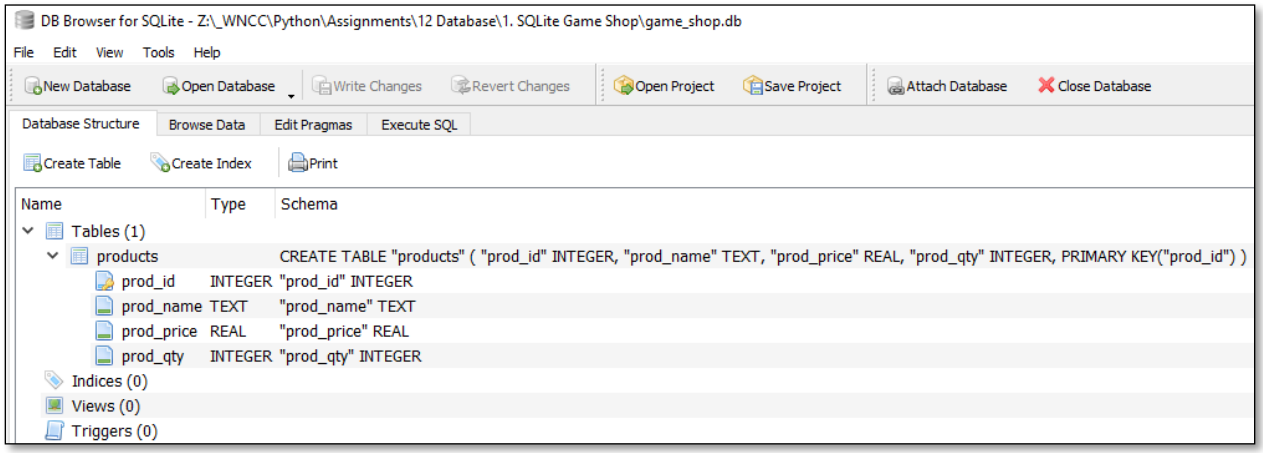
1. Double Click **SQLiteDatabaseBrowserPortable.exe**

2. Click **New Database.**
3. Filename: **game_shop.db →** Click **Save.**
4. Edit table definition
    a. Table: **products**
5. Fields:
    a. Click **Add**: Name: **prod_id**
    b. Click **Add**: Name: **prod_name**
    c. Click **Add**: Name: **prod_price**
    d. Click **Add:** Name: **prod_qty**
6. Make the setting changes as shown below. Notice the SQL code is created automatically. This is same code you would use to create this database and table in Python. SQL is SQL wherever it is used.



7. Click **OK** to create your table.

You should see the following result.



8. The changes you made are only in memory. To commit the changes to the database file, Click **Write Changes**.
9. Click **Close Database** to disconnect from the database.
10. Close DB Browser for SQLite.

Tutorial 2: Create a Database with Python

Explanation: How to Connect to an SQLite Database in Python

When we connect to an SQLite database, we are accessing data that resides in a file on our local computer.

1. To use SQLite, import **sqlite3**.

```
import sqlite3
```

2. Create a connection object using the **connect()** method and pass the name of the database file you want to access. If there is a file with that name, it will open that file. Otherwise, Python will create a file with the given name.
   The connection object represents the database in memory.

```
connection = sqlite3.connect("game_store_1.db")
```

3. A cursor object is used to send SQL and other commands to the database file.

```
cursor = connection.cursor()
```

4. Always close the cursor and connection to the database when the operation is done.

```
cursor.close()
connection.close()
```

Do: sqlite_tutorial_1_create_database.py

Please create the following Python program. **sqlite_tutorial_1_create_database.py**

Please keep copies of each tutorial as you go in case something goes wrong. Use the name shown in the tutorial screenshot.

```python
"""
    Name: sqlite_tutorial_1_create_database.py
    Author: William Loring
    Created: 01/05/22
    Use SQLite with Python
"""
# Import sqlite3 database library
import sqlite3


class GameShop():

    """
    Represents a game shop management system.

    Attributes:
        database (str): The name of the database file.
    """

    def __init__(self, database: str):
        """
        Initializes the GameShop class.

        Parameters:
            database (str): The name of the database file.
        """

        # Assign the provided database name to the class attribute
        self.database = database

        # Create a connection object which will connect to or
        # create an SQLite database file
        connection = sqlite3.connect(self.database)

        # Create a cursor object to interact with the database
        cursor = connection.cursor()
        print("Connected to the database")

        # Always close the cursor and connection to prevent resource leak
        cursor.close()
        connection.close()
        print("cursor and connection closed")


# Create an instance of the GameShop class with the
# database file name "game_shop_1.db"
game_shop = GameShop("game_shop_1.db")
```

Example run:

```
Connected to the database
cursor and connection closed
```

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.